

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования «Казанский национальный исследовательский
технический университет им. А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации

Кафедра Систем Информационной Безопасности

Лабораторная работа №1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ,
синхронизация потоков»

Выполнил:
студент гр. 4404
Сафин К. Р.

Казань, 2026

Цель работы

Изучение основ параллельного программирования для систем с общей памятью средствами OpenMP: создание параллельных областей и идентификация потоков, распределение итераций цикла между потоками (типы расписания schedule), примитивы синхронизации потоков.

В ходе работы выполнены задачи 1–3 методических указаний «Параллельное программирование на OpenMP». Работа выполняется на языке C++, в качестве средства распараллеливания используется технология OpenMP:

1. `#pragma omp parallel` создаёт команду потоков (модель fork-join);
2. `#pragma omp parallel for` распределяет итерации между потоками;
3. `#pragma omp critical` и `omp_lock_t` реализуют критическую секцию и блокировки;
4. `#pragma omp barrier` — точку синхронизации;
5. `#pragma omp flush` — сброс разделяемых переменных в память.

Задание

Задача 1. Написать программу, в которой каждый поток печатает свой идентификатор, количество потоков всего и строку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов, инициализированный так, что значение элемента равно его порядковому номеру. Создать результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива:

$$b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0$$

Запустить программу с 8 потоками при различных типах распределения работ.

Задача 3. Модифицировать задачу 1 так, чтобы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения; необходимо реализовать и продемонстрировать как можно больше из них.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, реализующие задачу целиком; код размещается в репозитории GitHub; по ходу выполнения делаются скриншоты.

Ход работы

Задача 1. Параллельная область и идентификация потоков

Программа считывает из командной строки число потоков N и устанавливает его вызовом `omp_set_num_threads(N)`. Директива `#pragma omp parallel` создаёт команду из N потоков — аналог модели `fork-join`: основной поток порождает дочерние, каждый из которых выполняет тело параллельной области. Внутри области каждый поток с помощью `omp_get_thread_num()` получает свой идентификатор, а с помощью `omp_get_num_threads()` — общее число потоков в команде, после чего печатает идентификатор, количество потоков и строку «Hello World». Вывод защищён критической секцией `#pragma omp critical`, чтобы строки разных потоков не перемешивались посимвольно. Порядок вывода при этом не фиксируется.

Фрагмент кода:

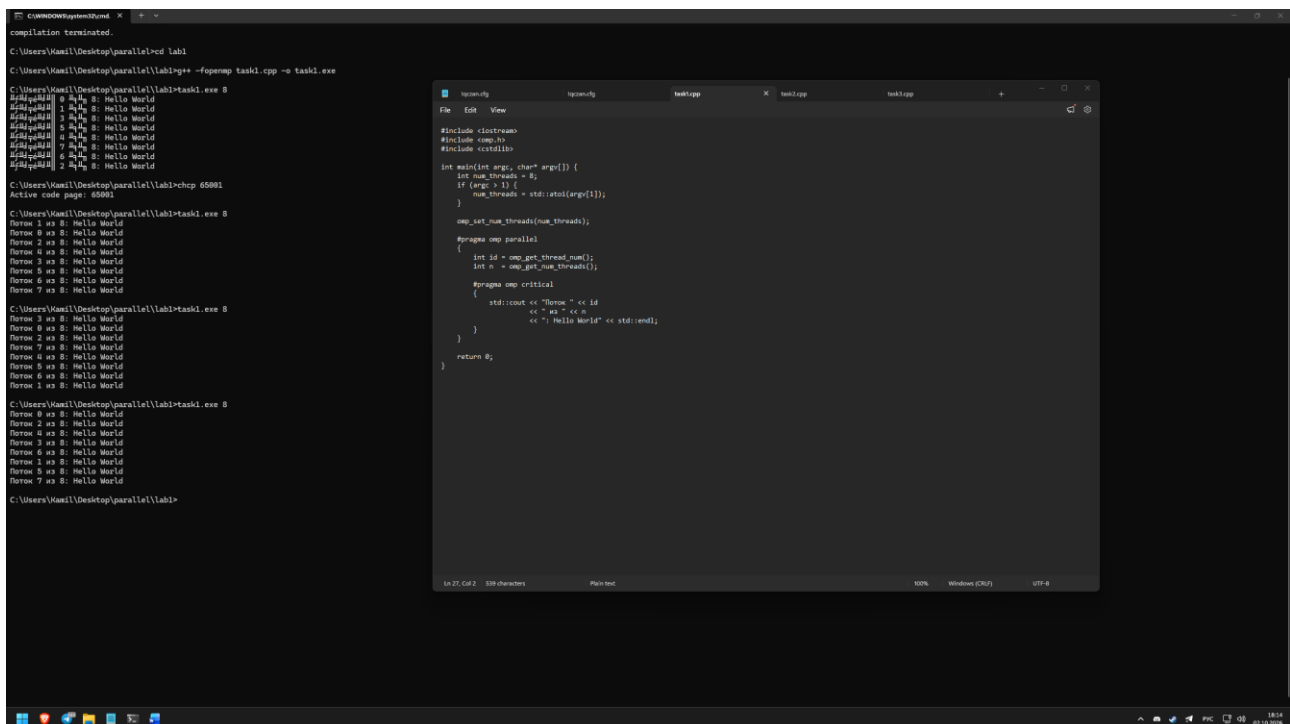
```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int n = omp_get_num_threads();
    #pragma omp critical
    {
        std::cout << "Поток " << id
                  << " из " << n
                  << ": Hello World" << std::endl;
    }
}
```

Команды запуска:

```
cd C:\Users\Kamil\Desktop\parallel\lab1\task1
g++ -fopenmp task1.cpp -o task1.exe
task1.exe 8
```

Ответ на вопрос задачи: вывод не всегда идентичен. Потоки выполняются параллельно, и порядок, в котором они достигают оператора вывода, определяется планировщиком операционной системы, а не программой. Множество строк одинаково во всех запусках (по одной от каждого из 8 потоков), но их последовательность может меняться. Это классический пример состояния гонки (race condition) на общем ресурсе — стандартном потоке вывода. Программа порядок не фиксирует, поэтому детерминированность не гарантируется.

На рисунке 1 показаны 3 запуска программы с 8 потоками. Видна команда запуска и полный вывод: каждый из 8 потоков напечатал свой идентификатор, общее число потоков и строку «Hello World». Набор строк одинаков, их порядок различен.



The image shows a Windows environment with a command prompt and a code editor. The command prompt displays the compilation and execution of a program named 'task1.exe'. The code editor shows the source code of 'task1.cpp', which uses OpenMP to create 8 parallel threads. Each thread prints its ID, the total number of threads (8), and 'Hello World'.

```
compilation terminated.
C:\Users\Vasil\Desktop\parallel\lab1>cd lab1
C:\Users\Vasil\Desktop\parallel\lab1>g++ -fopenmp task1.cpp -o task1.exe
C:\Users\Vasil\Desktop\parallel\lab1>task1.exe 8
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
C:\Users\Vasil\Desktop\parallel\lab1>chcp 65001
Active code page: 65001
C:\Users\Vasil\Desktop\parallel\lab1>task1.exe 8
Thread 1 is 8: Hello World
Thread 0 is 8: Hello World
Thread 2 is 8: Hello World
Thread 4 is 8: Hello World
Thread 3 is 8: Hello World
Thread 5 is 8: Hello World
Thread 6 is 8: Hello World
Thread 7 is 8: Hello World
C:\Users\Vasil\Desktop\parallel\lab1>task1.exe 8
Thread 3 is 8: Hello World
Thread 0 is 8: Hello World
Thread 2 is 8: Hello World
Thread 7 is 8: Hello World
Thread 4 is 8: Hello World
Thread 5 is 8: Hello World
Thread 6 is 8: Hello World
Thread 1 is 8: Hello World
C:\Users\Vasil\Desktop\parallel\lab1>task1.exe 8
Thread 0 is 8: Hello World
Thread 2 is 8: Hello World
Thread 4 is 8: Hello World
Thread 3 is 8: Hello World
Thread 6 is 8: Hello World
Thread 1 is 8: Hello World
Thread 5 is 8: Hello World
Thread 7 is 8: Hello World
C:\Users\Vasil\Desktop\parallel\lab1>
```

```
#include <iostream>
#include <omp.h>
#include <cstdlib>

int main(int argc, char* argv[]) {
    int num_threads = 8;
    if (argc > 1) {
        num_threads = atoi(argv[1]);
    }
    omp_set_num_threads(num_threads);
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int n = omp_get_num_threads();
        #pragma omp critical
        {
            std::cout << "Thread " << id
                      << " is " << n
                      << ": Hello World" << std::endl;
        }
    }
    return 0;
}
```

Рис. 1. Код 1-й программы, её вывод

Задача 2. Обработка массива при различных типах распределения работ

Программа создаёт массив a из N элементов ($N = 16000$ передаётся через командную строку), где $a[i] = i$, и вычисляет результирующий массив b по формуле:

$$b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0$$

для внутренних индексов $i = 1 \dots N-2$. Крайние элементы $b[0]$ и $b[N-1]$ в условии не определены и копируются из исходного массива.

Обработка внутренних элементов распределяется между 8 потоками директивой `#pragma omp parallel for` с условием `schedule(runtime)`. Тип расписания и размер блока читаются из переменной окружения `OMP_SCHEDULE`, что позволяет перебирать варианты без перекомпиляции программы:

1. `static` без блока: диапазон итераций делится между потоками непрерывными равными частями;
2. `static` с блоком `chunk`: блоки фиксированного размера раздаются потокам циклически;
3. `dynamic`: потоки захватывают блоки из общей очереди по мере освобождения, размер блока по умолчанию 1;
4. `guided`: размер выдаваемого блока уменьшается по мере обработки массива;
5. `runtime`: тип расписания и размер блока читаются из `OMP_SCHEDULE`.

Фрагмент кода:

```
#pragma omp parallel for schedule(runtime)
for (int i = 1; i < n - 1; ++i) {
    b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
}
```

Гонки данных отсутствуют: каждая итерация записывает только в свой элемент $b[i]$, массив a не изменяется. Поэтому результирующий массив не зависит от типа расписания (при $a[i] = i$ получаем $b[i] = i$).

Команды запуска:

```
cd C:\Users\Kamil\Desktop\parallel\lab1\task2
```

```
g++ -fopenmp task2.cpp -o task2.exe
```

```
set OMP_SCHEDULE=static
```

```
task2.exe 8 16000
```

```
set OMP_SCHEDULE=dynamic
```

```
task2.exe 8 16000
```

```
set OMP_SCHEDULE=guided
```

```
task2.exe 8 16000
```

```
set OMP_SCHEDULE=static,1000
```

```
task2.exe 8 16000
```

```
set OMP_SCHEDULE=dynamic,100
```

```
task2.exe 8 16000
```

На рисунке 2 приведён общий скриншот с пятью последовательными запусками программы с 8 потоками при различных типах распределения работ, а также исходный код программы.

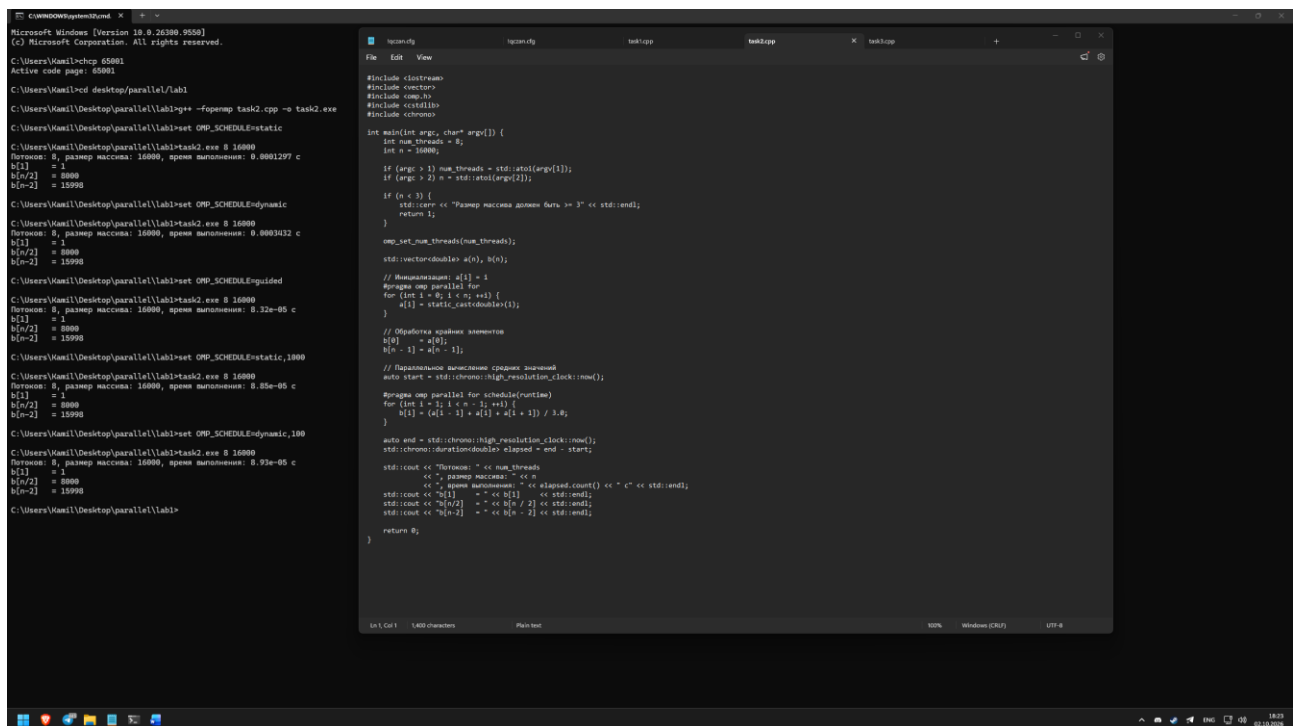


Рис. 2. Код 2-й программы, 5 запусков с 8 потоками при различных типах распределения работ

Из скриншота видно, что во всех пяти запусках получены одинаковые значения $b[i]$:

$$b[1] = 1$$

$$b[n/2] = 8000$$

$$b[n-2] = 15998$$

Различаются только время выполнения и статистика распределения итераций по потокам. Это подтверждает, что при независимых итерациях результат не зависит от типа расписания. Для задачи такого объёма (16000 элементов) время порядка 100 микросекунд, и разница между schedule лежит в пределах погрешности измерений. Динамическое распределение (dynamic) на равномерной нагрузке обычно медленнее статического из-за накладных расходов на синхронизацию при захвате блоков.

Задача 3. Вывод идентификаторов потоков в обратном порядке

Для фиксации порядка вывода необходимы средства синхронизации. В работе реализовано пять способов. Выбор способа осуществляется аргументом командной строки.

Способ 1. Барьер в цикле по turn. Потоки синхронно проходят по значениям $turn = N-1, N-2, \dots, 0$. На каждой итерации поток, чей идентификатор совпадает с текущим turn, печатает строку, после чего все потоки встречаются на барьере (`#pragma omp barrier`).

Способ 2. Критическая секция и busy-wait. Общая переменная turn инициализируется значением $N-1$. Каждый поток в цикле проверяет `turn == id`; при совпадении — печатает строку и уменьшает turn внутри критической секции. Для корректной видимости turn используется `#pragma omp flush(turn)`.

Способ 3. Блокировки OpenMP (`omp_lock_t`). Аналогичен способу 2, но защита turn выполняется не критической секцией, а функциями `omp_set_lock` и `omp_unset_lock`.

Способ 4. Флаги и ожидание соседнего потока. Каждый поток ожидает, пока следующий по номеру поток ($id+1$) не установит флаг `printed[id+1] = true`. После этого печатает свой идентификатор и устанавливает собственный флаг.

Способ 5. Барьер перед проверкой. Цикл `for (i = N-1; i >= 0; --i)`, перед каждой проверкой — `#pragma omp barrier`. Поток с `id == i` печатает строку.

Фрагмент кода:

```
for (int turn = n - 1; turn >= 0; --turn) {  
    if (id == turn) {  
        #pragma omp critical  
        std::cout << "Метод 1: Поток " << id << std::endl;  
    }  
    #pragma omp barrier  
}
```

Команды запуска:

```
cd C:\Users\Kamil\Desktop\parallel\lab1\task3
```

```
g++ -fopenmp task3.cpp -o task3.exe
```

```
task3.exe 8 1
```

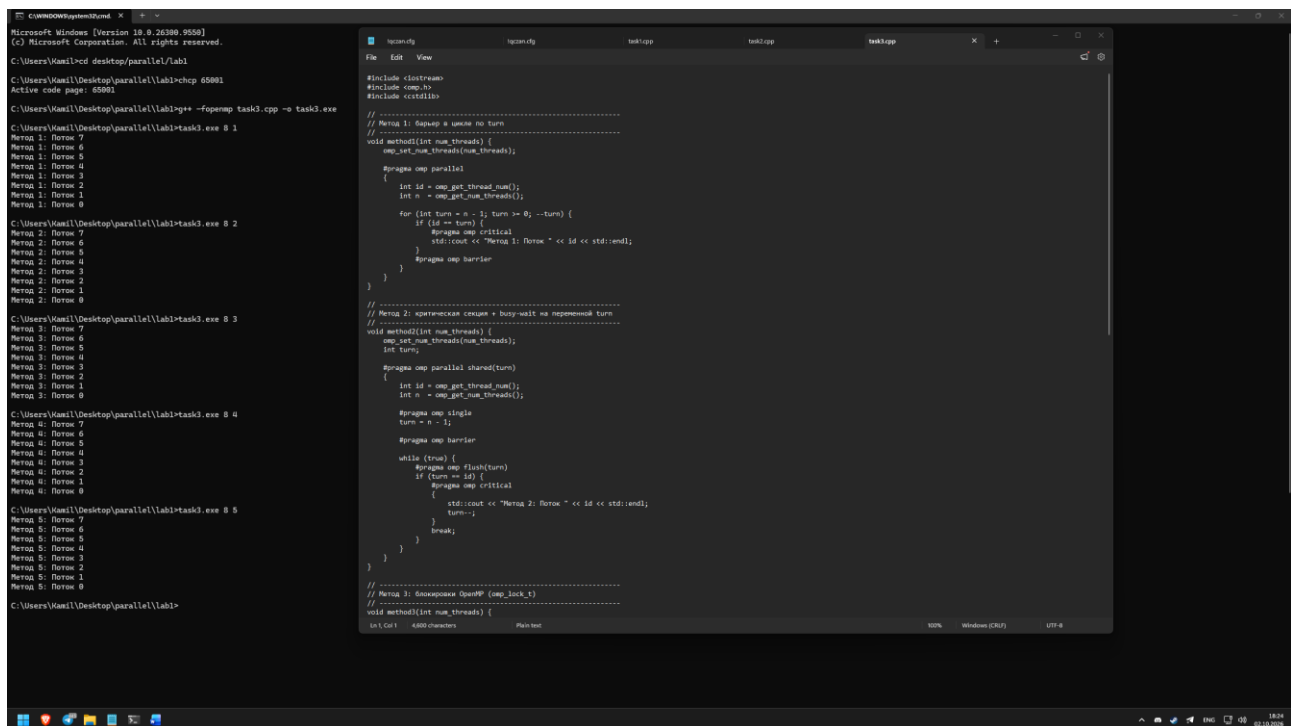
```
task3.exe 8 2
```

```
task3.exe 8 3
```

```
task3.exe 8 4
```

```
task3.exe 8 5
```

На рисунке 3 приведён общий скриншот с пятью последовательными запусками программы с 8 потоками, соответствующими пяти разным способам синхронизации, а также исходный код программы.



```
Microsoft Windows [Version 10.0.26300.9500]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Kamil\Desktop\parallel\lab1>
C:\Users\Kamil\Desktop\parallel\lab1>g++ -fopenmp task3.cpp -o task3.exe
C:\Users\Kamil\Desktop\parallel\lab1>task3.exe 8 1
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0

C:\Users\Kamil\Desktop\parallel\lab1>task3.exe 8 2
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0

C:\Users\Kamil\Desktop\parallel\lab1>task3.exe 8 3
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0

C:\Users\Kamil\Desktop\parallel\lab1>task3.exe 8 4
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0

C:\Users\Kamil\Desktop\parallel\lab1>task3.exe 8 5
Поток 7
Поток 6
Поток 5
Поток 4
Поток 3
Поток 2
Поток 1
Поток 0

C:\Users\Kamil\Desktop\parallel\lab1>
```

```
#include <iostream>
#include <omp.h>
#include <stdio.h>

// Метод 1: барьер и unique no turn
// Метод 2: критическая секция + busy-wait на следующем turn
// Метод 3: глобальная OpenMP lock_t

void method1(int num_threads) {
    omp_set_num_threads(num_threads);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int n = omp_get_num_threads();

        for (int turn = n - 1; turn >= 0; --turn) {
            if (id == turn) {
                #pragma omp critical
                std::cout << "Метод 1: Поток " << id << std::endl;
            }
            #pragma omp barrier
        }
    }
}

// Метод 2: критическая секция + busy-wait на следующем turn
void method2(int num_threads) {
    omp_set_num_threads(num_threads);
    int turn;

    #pragma omp parallel shared(turn)
    {
        int id = omp_get_thread_num();
        int n = omp_get_num_threads();

        #pragma omp single
        turn = n - 1;

        #pragma omp barrier

        while (true) {
            #pragma omp flush(turn)
            if (turn == id) {
                #pragma omp critical
                {
                    std::cout << "Метод 2: Поток " << id << std::endl;
                    turn--;
                }
                break;
            }
        }
    }
}

// Метод 3: глобальная OpenMP lock_t
void method3(int num_threads) {
    omp_set_num_threads(num_threads);
    omp_lock_t lock;
```

Рис. 3. Код 3-й программы и пять запусков с 8 потоками при различных способах синхронизации

Из скриншота видно, что во всех пяти запусках идентификаторы печатались строго в обратном порядке: 7, 6, 5, 4, 3, 2, 1, 0.

Это подтверждает корректность каждого из пяти реализованных способов. Способы 2 и 3 используют активное ожидание (busy-wait), что создаёт нагрузку на процессор, но требует меньше синхронных операций. Способы 1 и 5 основаны на барьерах и дают наиболее чистую синхронизацию. Способ 4 имеет линейную структуру ожидания «цепочкой» и не требует явных барьеров.